21차시 — 큰 수와 분수, 화면에 보이는 소수

설명

큰바른수는 아주 큰 정수를 정확하게 다룹니다. 큰 정수의 글 표기를 생성 함수에 명시적으로 넘깁니다. 나눔수는 분자와 분모를 유지하는 정확한 분수입니다. 일반 소수 계산과 같은 표현이라고 생각하지 마세요.

현재 알파3 화면은 일반 소수를 소수점 아래 최대 여섯 자리까지 반올림합니다. 정확히 중간이면 마지막 자리가 짝수가 되는 쪽을 택합니다. 계산값·저장값을 여섯 자리로 자르는 규칙은 아닙니다. 큰정수와 분수는 정확한 표현을 유지하고, 글은 숫자처럼 보여도 그대로입니다.

예상

21A 원문을 실행하기 전에 네 줄의 결과를 적어 보세요.

- 90071992547409 93에 1을 더하면?
- 1/3과 1/6을 더하면?
- 1/128은 소수로 얼마이며, 여섯 자리 표시에서는 어떻게 될까요?
- 따옴표로 감싼 "0.0078125"도 반올림될까요?

실행

셈그림에서 새 v25 문서를 만들고 원문을 붙여 넣어 실행하거나, CLI에서 다음처럼 실행합니다.

```
./bin/ddonilang.exe 실행 21A_정확한수.ddn --source-profile v1-core-v25 --마디수 1
```

CLI는 원문 파일을 둔 실제 경로를 지정하세요. 셈그림 첫걸음에서 21A를 고를 수도 있습니다. 21~28차시 학생용 PDF에는 이 보충 활동이 함께 들어 있습니다.

수정

큰수의 시작 값을 9007199254740994로 바꿔 보세요. 두 분수를 1/4과 1/8로 바꾸고 합을 예상하세요. 마지막 글을 "1/2"로 바꿔도 분수 계산이 되는지 확인하세요.

이유 설명

정답은 차례대로 9007199254740994, 1/2, 0.007812, 0.0078125입니다. 1/128의 정확한 소수값은 0.0078125입니다. 여섯 번째 자리 2가 짝수라 화면 표시는 0.007812입니다. 마지막 값은 글이므로 바뀌지 않습니다.

친구에게 "계산을 정확하게 하는 것"과 "읽기 편하게 표시하는 것"의 차이를 설명해 보세요.

22차시 — 숫자만 같으면 같은 양일까?

설명

2@m는 길이 2미터입니다. 200@cm와 같은 길이지만 2@s와는 다른 차원입니다. 단위는 화면에 붙인 장식이 아니라 계산에서 확인하는 값의 성질입니다. 이미 길이로 결속한 이름에는 지원되는 길이 단위를 다시 넣을 수 있습니다. 시간을 넣으면 거부합니다.

이 실습에서는 기존 길이·시간·넓이만 씁니다. 새로운 온도 환산이나 모든 단위의 지원을 가정하지 않습니다.

예상

22A를 읽고 다음을 종이에 적습니다.

1. 250센티미터는 몇 미터인가요?
2. 300센티미터와 400센티미터를 곱한 넓이는 몇 제곱미터인가요?
3. 넓이와 길이를 비교할 때 왜 숫자만 보아서는 안 될까요?

실행

첫걸음에서 22A를 골라 ▶를 누릅니다. 예상한 2.5@m, 12@m^2와 비교합니다. 22B는 일부러 틀린 원문입니다. 거리 이름에 시간을 넣으므로 오류가 나야 합니다. 오류 위치가 어느 문장을 가리키는지 읽어 보세요. 실패를 정상적인 결과값으로 받아들이지 않습니다.

수정

250을 125로 바꾸고 다시 실행합니다. 이어서 300과 400을 50과 200으로 바꾸세요. 결과는 1.25미터, 1제곱미터여야 합니다. 22B의 시간을 300@cm로 고치면 왜 실행할 수 있는지 설명합니다.

이유 설명

센티미터에서 미터로 길이를 바꾸면 100으로 나눕니다. 넓이는 두 길이를 곱하므로 300cm × 400cm = 3m × 4m = 12m²입니다. 단위를 빼 버린 120000이라는 숫자만으로는 어떤 넓이인지 알 수 없습니다. 단위 오류를 없애려고 단위를 삭제하는 것과 올바른 단위로 고치는 것은 다릅니다.

23차시 — 시작할 때 준비한 값은 언제 바뀔까?

설명

최상위 채비는 새 세계를 만들 때 자리를 준비합니다. 초기식은 처음 한 번 평가합니다. (매마디)마다 본문은 그 뒤 마디마다 실행합니다. 이 두 일을 섞으면 합계가 늘어야 할 때 시작값으로 계속 돌아갈 수 있습니다.

```
채비 { 합계:수 := 10. }.
(매마디)마다 { 합계 <- 합계 + 2. 합계 보여주기. }.
```

이 설명은 최상위 채비의 수명입니다. 셈씨 안의 지역 이름, 일반 정의, 지속 파생 관계를 모두 같은 규칙으로 취급하지 않습니다.

예상

23A는 3마디를 실행합니다. 세 줄은 12, 12, 12일까요, 12, 14, 16일까요? 합계의 시작값과 매마디 할 일을 다른 색으로 표시해 보세요.

실행

23A를 실행해 세 줄을 확인합니다. 한 마디 진행도 사용해 봅니다. 화면의 연속 실행 속도와 세계가 몇 마디 진행했는지는 다른 것입니다. 시간이 잠깐 흘렀다는 이유만으로 결과를 예상하지 말고 실제 마디 수를 확인하세요.

수정

초기값10을20으로, 더하는 값2를3으로 바꾸면 예상은23,26,29입니다. 다시 실행해 확인합니다. 이어서 시작값은 그대로 두고 매마디 본문에 합계 <- 10.을 먼저 넣어 보세요. 왜 매번12가 되는지 설명합니다. 이것은 초기화 규칙이 바뀐 것이 아니라 원문이 매번 다시결속한 것입니다.

이유 설명

저장한 상태를 복원하면 현재 합계를 이어받습니다. 처음부터 다시 시작하면 초기식으로 새 세계를 만듭니다. 따라서 합계14를 저장했다면 이어서 한 마디는16이고, 처음부터 한 마디는12입니다. 실제 저장 실험은27차시에서 합니다. 같은 화면을 다시 여는 것과 새 실행을 시작하는 것은 같지 않습니다.

24차시 — 표의 칸을 골라 바꾸기

설명

Basic Tensor는 같은 씨의 값을 정해진 모양으로 다룹니다. 이 수업에서는 2행2열의 수 표를 씁니다. 수학의 고급 텐서·계량이나 모든 다차원 연산을 배우는 차시는 아닙니다.

```
표 <- (차림=차림[차림[1,2],차림[3,4]], 원소씨=셈수, 모양=(2,2)) 텐서로.
표[0,1] <- 8.
열 <- 표[:,1..2].
```

칸 번호는0부터 시작합니다. [0,1]은 첫 행의 두 번째 칸입니다. :는 해당 축 전체, 1..2는1번부터 끝2번 직전까지입니다. 이 선택은 두 번째 열만 남깁니다.

예상

24A에서 옛표 <- 표. 다음에 표의 한 칸을 바꿉니다. 옛표와 새 표에서 읽은 값은 같을까요? 선택한 열은 몇 행 몇 열일까요?

실행

24A를 실행하면 2, 8, 8, 4가 나옵니다. 첫 두 줄은 옛 값과 새 값, 다음 두 줄은 선택한 열의 위·아래 값입니다. 원문의 네 출력문과 결과 네 줄을 짝지어 보세요.

수정

8을10으로 바꿔 네 결과를 예상합니다. 선택을 표[:,0..1]로 바꾸면 첫 번째 열이므로1과3이 나옵니다. 표[2,0]처럼 없는 행을 읽어 오류 위치를 확인한 뒤0이나1로 고칩니다. 없는 칸을 조용히0으로 채우는 결과를 기대하지 마세요.

이유 설명

옛표는 갱신 전 값을 유지합니다. 새 표를 바꾸는 것과 옛 값을 관찰하는 일을 구분해야 비교 실험이 가능합니다. 축 순서와 선택 범위를 그림으로 그려 친구에게 설명해 보세요. 같은 숫자 네 개라도 2×2와1×4는 같은 모양이 아닙니다.

25차시 — 지금과 과거를 같은 화면에서 비교하기

설명

공통 마디는 세계의 현재 마디를, 고정 마디는 정해 둔 마디를, 상대 마디는 현재에서 정해진 차이만큼 떨어진 마디를 봅니다. 이는 화면 관찰의 시간 선택입니다. 과거를 본다고 세계를 되감거나 입력 기록이 새로 생기지 않습니다.

> (번호=0) 표준시간.고정마디
> (차이=-1) 표준시간.상대마디

시간 인자는 보기 관계에 붙입니다. 이 차시에서는 제품에 보존된 공통·고정·상대 보기만 사용합니다. 없는 과거를 현재 그림으로 대신 보여 주면 비교가 아니라 잘못된 관찰입니다.

예상

25A의 가로 위치는 마디마다1씩 늘어납니다. 3마디 후 현재 공의 x는3입니다. 고정0번 마디 그림의 x는1일까요0일까요? 0번 마디는 첫 마디 처리를 마친 상태라 x=1입니다. 25B의 직전 마디는 x=2입니다.

실행

첫걸음25A를 실행하고 으뜸과 곁을 비교합니다. 이어서25B를 실행합니다. 고정 그림은 그대로이고 상대 그림은 현재와 함께 바뀝니다. 25C는 같은 구분을 결과표로 보여 줍니다. 보개 이름이 여럿이면 원문에서 선언 한짜임을 명시적으로 선택해야 합니다.

수정

고정 번호0을1로 바꿔3마디 실행하면 고정 그림은x=2입니다. 아직 만들지 않은9번으로 바꾸면 오류가 나야 합니다. 상대 차이-1을0으로 바꾸면 현재와 같은 마디를 봅니다. 실패한 보기 선택이 세계를 진행시켰다고 취급하지 마세요.

이유 설명

곁을 닫거나 다시 여는 것은 관찰 조작입니다. 세계의 x를 다시 계산하는 명령이 아닙니다. "고정 그림", "직전 그림", "입력 재생"의 차이를 각각 한 문장으로 설명해 보세요. 지원되는 이력은 무한하지 않으며, 이 수업은 무한한 과거 저장을 보장하지 않습니다.

26차시 — 행동, 눌린 상태, 반응의 순서

설명

키는 호스트에서 논리 행동으로 바뀝니다. 세계는 그 마디의 InputSnapshot을 읽습니다. "눌려 있다"는 상태와 "방금 눌렀다"는 사건은 다릅니다. 같은 마디의 사건은 순서가 있습니다. 반응은 사건마다 앞 사건의 반영 상태를 보고, 매마디 처리는 그 뒤 결과를 봅니다.

알림씨는 형 있는 사건을 담고, 임자의 움직씨가 반응합니다. 연결은 원문이 명시합니다. 이름이 같다고 자동 연결하지 않습니다. 현재 입력사상.알림결속과 받는자리 철자는 잠정 표면입니다. 이 수업은 명시 연결 방식을 배우는 것이지 이 이름의 영구 확정을 뜻하지 않습니다.

예상

26A는20A와 같은 원문·계약을 재사용합니다. 시작값2에서 더함→곱함은6, 곱함→더함은5입니다. 같은 사건 두 개를 순서 없이 모아 두면 왜 결과를 설명할 수 없을까요?

실행

26A를 고르고 학습 안내의 논리 입력 계약 선택을 누른 뒤 ▶로 실행합니다. 왼쪽·오른쪽 키를 눌러 사건과 결과표를 읽고 일시정지합니다. 키를 뗀 사건도 기록되므로 직접 누르기 결과가 언제나6 또는5라고 단정하지 않습니다. 한 마디의 두 순서 비교는 고정 입력 검사로 따로 확인합니다.

수정

종이에 더함1을3으로 바꾼 두 순서의 결과10과7을 계산합니다. 실제 원문을 편집하면 원문 해시가 달라지므로 기존 입력 계약을 몰래 그대로 쓸 수 없습니다. 셈그림에서 새 계약을 명시적으로 선택·결속한 뒤 새로 실행하세요. 기존 파일 재생과 새 원문 실험을 섞지 않습니다.

이유 설명

원자 실패는 "마디 일부만 성공한 것처럼 남기지 않는다"는 뜻입니다. 두 번째 사건 처리에서 실패하면 첫 사건이 바꾼 값도 그 마디의 정상 결과로 게시하지 않습니다. 반응이 끝났다고 사건을 목록에서 소비해 조회에 안 보이게 하지 않습니다. 화면 속도·키 배치를 바꾸는 것과 세계에 준 사건 순서를 바꾸는 것의 차이를 설명해 보세요.

27차시 — 원문 저장, 상태 복원, 입력 재생은 다르다

설명

원문 저장은 내가 쓴 프로그램을 보존합니다. 실행 상태 저장은 원문과 함께 현재 세계·마디·실제로 처리한 입력 상태를 보존합니다. 입력 기록 재생은 시작 조건과 완전한 입력 기록을 다시 적용합니다. 화면이 비슷하다고 이 세 가지가 같은 기능이 되는 것은 아닙니다.

이번에는23A의 계산을 뿌리 파일과 실행 움직씨가 있는 작은 작품으로 작성합니다. 현재 실행 상태 저장은 명시 뿌리 작품에서 지원합니다. 단일 원문이 저절로 영구 작품이 되지는 않습니다. 저장 후 초기식10으로 돌아가면 이어서 계산한 것이 아닙니다.

예상

한 마디 후12, 두 마디 후14입니다. 여기서 저장했다고 합시다. 복원 후 한 마디는16이고, 처음부터 한 마디는12입니다. 두 상황을 표로 적고 왜 다른지 설명합니다.

실행

27A를 고르고 작품 자료에서 정체 만들기를 명시적으로 선택한 뒤 실행합니다. 일시정지하고 처음부터 시작해 한 마디 진행으로 합계12, 다음 마디14를 확인합니다. 작품 자료에서 현재 실행 상태 저장을 선택하고 파일을 내보냅니다. 재열기로 복원해 원문·마디·합계가 같음을 확인한 뒤 한 마디를 진행하면16입니다. 정체 만들기 없이 저장을 시도하면 거부되는 것이 정상입니다.

상태 파일은 원문 또는 입력 테이프 파일과 다릅니다. 엉뚱한 파일을 넣었을 때 조용히 빈 세계로 바뀌는 결과를 정상으로 받아들이지 마세요.

수정

새 실행의 시작값을20으로 고치고 저장 전 원문과 비교합니다. 원문이 다른데 기존 상태를 같은 실행이라고 주장할 수 있을까요? 기존20A/26A에서는 입력 기록 저장·재생을 따로 해 보고, 상태 복원과 어떤 정보가 다른지 설명합니다.

이유 설명

복원은 저장 시점에서 이어 가기이고 재생은 입력을 다시 적용하기입니다. 상태는 있지만 전체 입력 이력이 없으면 완전한 Tape 재생을 만들 수 없습니다. 현재 브라우저 저장 검사가 통과했다는 사실은 전원 손실 등 모든 복구 상황의 내구성 보장이 아닙니다. 글자 입력 같은 미지원 기능의 기록이 되는 것처럼 설명하지 않습니다.

28차시 — 세금을 내는 사람과 부담하는 사람

설명

종합 작품은 기존 작은 시장 실험실입니다. 원문에서 정의한 셈씨로 복리를 계산하고, 시장 가정을 바꾸며, 으뜸·겹·곁으로 결과를 비교합니다. 세금을 판매자가 납부한다고 경제적 부담을 모두 판매자가 지는지 먼저 생각해 봅니다.

이 작품은 선형 수요·공급의 작은 모형이지 실제 시장 예측이 아닙니다. 수요는 Qd=100-5Pc, 공급은 Qs=20+3Ps, 소비가격과 생산가격 차이는 세금입니다. 물건마다 같은 반응을 보인다는 가정이 들어 있습니다.

예상

세금0일 때 균형 가격10, 수량50입니다. 세금8이면 소비가격13, 생산가격5, 수량35입니다. 소비자 부담은3, 생산자 부담은5이며 합은8입니다. "판매자가 납부하므로 부담도8"이라는 예상과 다른가요?

실행

28A를 실행합니다. 으뜸의 세전 수요·공급, 겹의 세후 공급과 두 가격, 곁 결과표를 읽습니다. 그림에서 읽은 수량·가격을 표와 비교합니다. 겹을 꺼도 계산값은 바뀌지 않습니다. '단리/복리' 상황은 다른 질문을 관찰하는 보기입니다.

수정

원문의 세금 := 8.을0,4,8로 바꾸고 매번 명시적으로 다시 실행합니다. 상황 단추는 초기값 원문을 바꾸며 자동으로 세계를 계산하지 않습니다. 세금4이면 소비가격11.5, 생산가격7.5, 수량42.5입니다. 바꾼 가정과 결과를 세 줄 표에 기록하세요.

복리 실험에서는 처음돈1000, 이자율0.05, 해수5로 단리1250과 복리1276.2815625를 비교합니다. 화면 여섯 자리 반올림과 계산값의 차이는21차시를 떠올립니다.

이유 설명

100-5(Ps+t)=20+3Ps에서 Ps=(80-5t)/8, Pc=(80+3t)/8입니다. 이 모형에서 부담 비율이3:5로 나뉘는 이유를 식과 그림 두 방법으로 설명합니다. 가정이 다른 시장에도 같은 부담 비율을 그대로 붙일 수 있을까요? 현실 자료와 모형을 구별해야 합니다.

마지막 활동: 원문의 한 셈씨, 한 상태/값, 한 보기 선언을 골라 각각의 역할을 설명하고, 잘못된 수정 한 가지를 만들어 진단을 읽은 뒤 고칩니다. 만들기→예상→실행→수정→설명이 한 흐름이 되어야 합니다.